ECOLE MAROCAINE DES SCIENCES DE L'INGENIEUR
Membre de HONORIS UNITED UNIVERSITIES

Compte rendu de TP

Entraîner, évaluer et sauvegarder un modèle de sentiment

Développement et déploiement d'applications intelligentes

Réalisé par
Othmane Moussawi
en binôme avec Ahmed Rayane Ramzi

Enseignant : A. Larhlimi

Année scolaire : 2026–2027
14 octobre 2026

Table des matières

1 Introduction **1**

2 Installation du kit **1**
2.1 Démarche 1
2.2 Réalisation 1
2.3 Interprétation 1

3 Jeu de démonstration **1**
3.1 Démarche 1
3.2 Réalisation 1
3.3 Interprétation 1

4 Entraînement **1**
4.1 Démarche 1
4.2 Réalisation 2
4.3 Interprétation 2

5 Évaluation et analyse des erreurs **2**
5.1 Démarche 2
5.2 Réalisation 2
5.3 Interprétation 3

6 Le vrai jeu : Allociné **3**
6.1 Démarche 3
6.2 Réalisation 3
6.3 Interprétation 3

1. Introduction

Ce TP transforme le cadrage du TP1 en un modèle concret. Le cas 3 (avis clients négatifs) relève du machine learning : il faut donc entraîner un classifieur de sentiment reproductible, l'évaluer correctement sur des données jamais vues, le sauvegarder pour l'interface du TP3, et comprendre ce qu'il se trompe à corriger. L'objectif n'est pas un modèle optimal, mais un modèle dont on sait ce qu'il vaut et pourquoi il se trompe.

2. Installation du kit

2.1 Démarche

Le kit `tp2-semaine2-kit` est installé dans le dépôt par le script `appliquer_tp2.py`, d'abord en simulation, puis pour de bon. Le script copie `src/`, `tests/test_model.py` et le notebook, et ajoute `!models/metrics.json` au `.gitignore`.

2.2 Réalisation

Contrôle C1, `pytest -q` juste après l'installation : **3 failed, 37 passed, 2 skipped**. L'énoncé attend 2 failed, 38 passed, 2 skipped. Les deux échecs attendus réclament `models/metrics.json`, absent avant l'étape C. Le troisième échec est le test d'historique Git (un seul auteur de commits), qui n'est pas lié au modèle.

2.3 Interprétation

Le surplus d'échec vient du dépôt, non du kit : le test d'historique Git exige deux auteurs, ce que la séance ne peut pas garantir tant que le second membre n'a pas poussé.

3. Jeu de démonstration

3.1 Démarche

Le script `src/make_demo.py` génère un jeu d'avis en français, déterministe grâce à une graine fixe, composé d'avis nettement positifs, nettement négatifs, mitigés et neutres.

3.2 Réalisation

```
python src/make_demo.py
```

Sortie obtenue : 290 avis écrits dans `data/demo_reviews.csv`, dont 145 positifs et 145 négatifs, avec 100 avis mitigés et 30 neutres. Le contrôle C2 est conforme.

3.3 Interprétation

Les avis mitigés ont une structure fixe : une première moitié d'un sentiment, un retournement (« mais finalement », « sauf que ensuite »), puis une seconde moitié de sentiment opposé. L'étiquette suit la seconde moitié. Cette structure annonce les erreurs de la partie 5.

4. Entraînement

4.1 Démarche

Le découpage est fait *avant* toute vectorisation, avec `test_size=0.2`, une stratification sur l'étiquette et `random_state=42`. Vectoriser d'abord laisserait le vocabulaire et les poids du TF-IDF connaître le jeu de test. Le `Pipeline` enferme le vectoriseur (`ngram_range=(1, 2)`,

`min_df=2`, `sublinear_tf=True`) et la régression logistique (`class_weight="balanced"`) dans un seul objet, sauvegardé en entier.

4.2 Réalisation

```
python src/train.py
```

Sortie obtenue :

```
Source : demo, 290 avis (145 négatifs)
232 avis d'entraînement, 58 de test
Vocabulaire : 316 termes
F1 macro sur le jeu de test : 0.879 (seuil 0.85)
```

Les fichiers produits sont `models/sentiment.joblib` (non versionné) et `models/metrics.json` (versionné). Le contrôle C3 est conforme, et seul `metrics.json` apparaît dans `git status`.

4.3 Interprétation

Ce qui se reconstruit par une commande n'entre pas dans Git. Seule la mesure, `metrics.json`, est une preuve qui ne se reconstruit pas sans réentraîner : c'est pour cette raison que la CI peut vérifier le seuil sans jamais posséder le modèle.

5. Évaluation et analyse des erreurs

5.1 Démarche

`src/evaluate.py` reprend exactement le même découpage (même graine, mêmes proportions) pour produire le rapport de classification et la matrice de confusion.

5.2 Réalisation

```
python src/evaluate.py
```

	prédit négatif	**prédit positif**
vérité négative	27	2
vérité positive	5	24

TABLE 1 – Matrice de confusion sur les 58 avis de test (jeu de démonstration).

Le script affiche 7 erreurs sur 58. Le contrôle C4 est conforme : toutes les erreurs portent sur des avis mitigés. Trois exemples relevés :

— vérité positive, prédit négatif (confiance 0,53) : « Taille trop petite et retour payant au déballage, sauf que ensuite produit conforme à la description, très satisfait. »

— vérité négative, prédit positif (confiance 0,56) : « Service impeccable et livraison en avance sur le papier, sauf que ensuite produit cassé à l'arrivée. »

— vérité négative, prédit positif (confiance 0,69) : « Service impeccable et livraison en avance les premiers jours, avant que notice incompréhensible et pièces manquantes. »

5.3 Interprétation

Les deux faux positifs sont le cas grave : un client mécontent traité en retard. Le mécanisme est celui d'un sac de mots, qui ne tient pas compte de l'ordre : « impeccable … puis cassé » et « cassé … puis impeccable » donnent un vecteur presque identique. Le modèle hésite d'ailleurs sur ces cas, avec des confiances entre 0,53 et 0,69. Corriger ce défaut demande une représentation sensible à l'ordre, comme des embeddings (semaine 9).

6. Le vrai jeu : Allociné

6.1 Démarche

Le même code est lancé sur 20 000 critiques de films étiquetées, téléchargées depuis Hugging Face, avec le même découpage. Le jeu est près de soixante-dix fois plus grand que celui de démonstration.

6.2 Réalisation

```
python src/train.py --allocine
python -m pytest -q
```

Sorties obtenues :

```
Source : allocine, 20000 avis (10204 négatifs)
16000 avis d'entraînement, 4000 de test
Vocabulaire : 149529 termes
F1 macro sur le jeu de test : 0.920 (seuil 0.85)
```

	Démonstration	Allociné
Avis	290	20 000
Vocabulaire	316 termes	149 529 termes
F1 macro	0,879	0,920

Table 2 – Comparaison des deux jeux.

Matrice de confusion sur 4 000 avis de test Allociné : 1 860 négatifs bien classés, 181 négatifs classés positifs, 137 positifs classés négatifs, 1 822 positifs bien classés (318 erreurs, soit 7,95

Avec `pytest -q`, on obtient **41 passed, 1 failed** : le test en échec est l'historique Git, qui attend le commit du second membre. Le contrôle C5 attend 42 passed après ce commit.

6.3 Interprétation

Le modèle n'a pas changé : ce sont les données qui ont changé. La mesure devient crédible, calculée sur 4 000 avis au lieu de 58. Sur Allociné, les négatifs manqués sont plus nombreux que les positifs manqués, à l'inverse de la démonstration.

Réponse

Modèle sauvegardé avec F1 macro 0,920 sur 4 000 avis. Erreurs principales : le sac de mots est insensible à l'ordre des phrases, ce qui pénalise les avis mitigés comme les textes qui se contredisent.

Conclusion

Le modèle est entraîné, évalué sur un jeu de test jamais vu, et sa mesure est versionnée. Le TP a aussi montré que les performances dépendent autant des données que du code : le même pipeline passe de 0,879 à 0,920 selon le jeu utilisé.